

Business Process Modeling

Preparazione all'orale

Risposte discorsive alle domande d'esame

MPB (6 cfu, 295AA)

Professore: Roberto Bruni

*Domande raccolte dagli appelli passati,
risposte pensate per un'esposizione orale completa*

Come usare questo documento

Questo documento raccoglie le domande realmente uscite agli orali di Business Process Modeling e, per ognuna, propone una risposta discorsiva e completa, scritta come la esporresti davanti al professore: prima l'intuizione e il *perché* serve il concetto, poi la definizione formale, infine le conseguenze e i collegamenti con il resto del corso.

L'indicatore $\times N$ accanto a ogni domanda dice in quante testimonianze distinte è comparsa: più è alto, più la domanda è probabile. Le domande sono raggruppate per argomento, nell'ordine in cui conviene ripassarle.

Ogni risposta è costruita con lo stesso schema:

- un box **Domanda** con il testo esatto e la frequenza;
- la risposta in prosa, con le formule integrate nel discorso;
- box **Definizione** e **Teorema** per i punti che vanno enunciati con precisione;
- un box **Da dire all'orale** col nocciolo da non dimenticare.

Indice

Come usare questo documento	i
1 Workflow net e soundness	1
1.1 Le tre condizioni di soundness	1
1.2 Il main theorem, con dimostrazione	2
1.3 La short-circuit net $N^\star$	3
1.4 Disegnare una rete che viola la proper completion	4
2 Invarianti	5
2.1 S-invariant: definizione, matematica, due teoremi	5
2.2 La proprietà fondamentale, con dimostrazione	7
3 Liveness e boundedness	8
3.1 Boundedness: definizione, notazione, legame con gli S-invariant	8
3.2 Tutto su liveness	9
3.3 Disegna una rete deadlock-free ma non live	10
3.4 Disegna una rete non bounded	11
4 Matrici e marking equation	12
4.1 Marking equation lemma, con dimostrazione	12
4.2 Matrice di incidenza	13
4.3 Parikh vector	13
5 Free-choice net	15
5.1 Rank theorem	15
5.2 Free-choice net e teorema di Commoner	16
5.3 Algoritmo per il massimo siphon non marcato	17
6 S-system e T-system	18
6.1 Proprietà fondamentale degli S-system	18
6.2 Boundedness e liveness degli S-system	19
7 S-coverability	20
7.1 S-coverability: definizione e relazioni	20
8 Grafi di raggiungibilità	22
8.1 Reachability graph vs coverability graph	22
9 EPC vs BPMN	24
9.1 Differenza tra BPMN ed EPC, con la notazione	24
10 Process mining e conformance	26
10.1 Footprint matrix	26
10.2 Alpha algorithm	27

11	Analisi quantitativa	29
11.1	Little’s Law	29
11.2	Cycle Time Efficiency	30

Capitolo 1

Workflow net e soundness

Questo è l'argomento più chiesto in assoluto: la soundness è il risultato centrale del corso, perché è la nozione di “correttezza” di un processo di business e viene ricondotta a proprietà (liveness e boundedness) che sappiamo verificare con gli strumenti standard sui Petri net.

1.1 Le tre condizioni di soundness

Domanda ×3

Enuncia le tre condizioni di soundness di un workflow net, con la loro notazione formale (option to complete, proper completion, no dead transition).

Partirei mettendo a fuoco *cosa* stiamo verificando. Un workflow net è un Petri net con una struttura particolare: un unico place iniziale i senza archi entranti, un unico place finale o senza archi uscenti, e ogni altro nodo che sta su almeno un cammino da i a o . La marcatura iniziale è sempre un solo token in i (avviare un caso), e “concludere il caso” significa avere un token in o .

Il punto chiave da sottolineare è che questa definizione è puramente strutturale: dipende solo dalla forma del grafo. Ci esclude già gli errori grossolani (nessun ingresso o uscita, ingressi multipli, nodi scollegati), ma non intercetta i difetti che emergono solo *eseguendo* il processo: transizioni che non si possono mai eseguire, token che restano “pendenti” nella rete dopo che il caso è teoricamente finito, oppure casi che entrano in un ciclo e non terminano più (livelock). La soundness serve esattamente a vietare questi difetti comportamentali.

Intuitivamente un processo è sound se: (1) non contiene task inutili, (2) ogni caso avviato può sempre essere portato a termine, (3) alla conclusione non restano item pendenti. Tradotto formalmente sui workflow net, le tre condizioni sono:

Definizione — Soundness di un workflow net

Un workflow net è sound se valgono tutte e tre:

1. No dead task: nessuna transizione è morta, cioè ogni transizione è eseguibile in qualche marcatura raggiungibile.

$$\forall t \in T. \exists M \in [i]. M \xrightarrow{t}$$

2. Option to complete: da ogni marcatura raggiungibile si può sempre arrivare a marcare o .

$$\forall M \in [i]. \exists M' \in [M]. M'(o) \geq 1$$

3. Proper completion: quando o è marcato, non resta nessun altro token nella rete.

$$\forall M \in [i]. M(o) \geq 1 \Rightarrow M = o$$

All'orale è efficace abbinare ogni condizione al difetto che vieta: la condizione 1 vieta i task *dead* (attività inutili, mai eseguibili); la condizione 2 vieta i *livelock* e i casi che non terminano più; la condizione 3 vieta i *token o le attività pendenti* dopo la fine del caso.

Vale la pena far notare la struttura dei quantificatori, perché è la stessa della liveness: la condizione 1 è un “ $\forall t \exists M$ ” (per ogni task *esiste* un modo di eseguirlo); la condizione 2 è un “ $\forall M \exists M'$ ”

(da *ogni* marcatura *esiste* una via verso *o*); la condizione 3 è un'implicazione “ $\forall M$, se *o* è marcato allora è l'unica cosa marcata”.

Attenzione — cosa NON dice l'option to complete

La condizione 2 non proibisce i cicli né mette un limite alle iterazioni: dice solo che da *qualunque* marcatura raggiungibile deve *esistere* una via verso *o*. Si assume implicitamente la fairness (una transizione non può essere rimandata all'infinito), quindi un ciclo va benissimo purché ci sia sempre un'uscita verso *o*.

Chiuderei dicendo che verificare la soundness “a forza bruta” significa costruire il reachability graph e controllare che ogni transizione etichetti almeno un arco e che ci sia un unico stato finale (un solo token in *o*) raggiungibile da ogni altro stato. Ma è costoso (state explosion) e può non terminare: per questo esiste il teorema principale, che riconduce tutto a liveness e boundedness.

Da dire all'orale — soundness

Tre condizioni: no dead task ($\forall t \exists M. M \xrightarrow{t}$), option to complete ($\forall M \exists M'. M'(o) \geq 1$), proper completion ($M(o) \geq 1 \Rightarrow M = o$). Vietano rispettivamente task inutili, non-terminazione, token pendenti.

1.2 Il main theorem, con dimostrazione

Domanda ×2

Enuncia e dimostra il main theorem: $N \text{ sound} \iff N^\star \text{ live e bounded}$ (in particolare la direzione $\text{sound} \Rightarrow \text{liveness}$).

Introdurrei il teorema spiegando l'*idea geniale* dietro. Verificare direttamente le tre condizioni di soundness è scomodo. La trasformazione consiste nel far ripartire il processo da capo: aggiungiamo alla rete una transizione reset che riporta il token da *o* a *i*. Così, invece di fermarsi alla fine, il processo può essere “giocato un numero qualsiasi di volte”, e possiamo applicare le proprietà di liveness e boundedness, che già sappiamo verificare con reachability graph, invarianti e tool come WoPeD.

Definizione — La rete $N^\star$

Dato un workflow net $N : i \rightarrow o$, la rete $N^\star$ si ottiene aggiungendo la transizione reset con $\bullet \text{reset} = \{o\}$ e $\text{reset}\bullet = \{i\}$. Ogni volta che il processo raggiunge *o*, reset rimette il token in *i* e il processo può ripartire.

Teorema — Main Theorem

$N \text{ è sound} \iff N^\star \text{ è live e bounded.}$

Per la dimostrazione conviene legare ciascuna delle tre condizioni di soundness alle due proprietà di $N^\star$, ragionando sul *perché* funziona:

- $N^\star \text{ live} \Rightarrow \text{no dead task}$ (cond. 1). Se $N^\star$ è live, ogni sua transizione può sempre essere riabilitata in futuro; in particolare nessuna transizione è dead. Quindi vale la condizione 1 su N .
- $N^\star \text{ bounded} \Rightarrow \text{proper completion}$ (cond. 3). Ragioniamo per assurdo. Se non valesse la proper completion, esisterebbe una marcatura raggiungibile con un token in *o* *più* qualche token residuo. A quel punto reset riporterebbe il token da *o* a *i*, ma i residui resterebbero: ripetendo il giro si accumulerebbero token senza limite, cioè $N^\star$ sarebbe unbounded. La boundedness

lo vieta, quindi la proper completion deve valere.

- live + bounded insieme $\Rightarrow$ option to complete (cond. 2). La liveness garantisce che reset sia sempre riabilitabile; ma reset scatta solo se o è marcato, quindi da ogni marcatura raggiungibile si può sempre arrivare a marcare o . Questa è esattamente la condizione 2.

Per la direzione che il professore chiede spesso in modo esplicito — $\text{sound} \Rightarrow N^\star \text{ bounded}$ — conviene mostrare la prova formale per assurdo, che usa il *monotonicity lemma* delle matrici. Se $N^\star$ fosse unbounded, per il criterio del coverability graph esisterebbe una coppia di marcature raggiungibili

$$i \xrightarrow{*} M \xrightarrow{*} M' \quad \text{con } M \subsetneq M'$$

cioè M' contiene strettamente M . Sia $L = M' - M > 0$ il “surplus” di token prodotto dal giro. Per monotonicità, dalla marcatura in cui N completa (un token in o) potremmo raggiungere

$$o + L$$

cioè un token in o più i token extra L : questo viola la proper completion, e quindi N non sarebbe sound. Per contrapposizione, $N \text{ sound} \Rightarrow N^\star \text{ bounded}$. ■

Chiuderei sottolineando la potenza del risultato: la soundness, che è una proprietà specifica dei workflow, viene ricondotta a due proprietà generali dei Petri net per cui esistono algoritmi e tool. E, come vedremo con le free-choice net, “liveness + boundedness” è addirittura decidibile in tempo polinomiale sulla classe di reti che i processi reali quasi sempre appartengono.

Da dire all'orale — main theorem

$N \text{ sound} \iff N^\star \text{ live e bounded}$. Legami: live $\Rightarrow$ no dead task; bounded $\Rightarrow$ proper completion (altrimenti reset accumula i residui $\rightarrow$ unbounded); live + bounded $\Rightarrow$ option to complete. Prova “sound $\Rightarrow$ bounded” per assurdo col monotonicity lemma: dal surplus $L = M' - M$ si raggiungerebbe $o + L$, violando la proper completion.

1.3 La short-circuit net $N^\star$

Domanda $\times 1$

Cos'è la short-circuit net $N^\star$ e a cosa serve?

Questa domanda è in pratica un pezzo della precedente, isolato. Risponderei: $N^\star$ (detta anche *short-circuit net*, perché “cortocircuita” l'uscita con l'ingresso) è la rete che si ottiene da un workflow net N aggiungendo un'unica transizione reset con pre-set $\{o\}$ e post-set $\{i\}$. Serve a trasformare un processo “one-shot” in un sistema ciclico: chiudendo l'anello da o a i , il processo diventa ripetibile all'infinito, e questo permette di applicare le nozioni di liveness e boundedness (che parlano del comportamento a lungo termine) a un oggetto che altrimenti termina dopo un solo caso.

C'è anche una ragione strutturale elegante da menzionare: aggiungere reset rende $N^\star$ strongly connected. Infatti, presi due nodi qualsiasi, esiste sempre un cammino orientato tra loro passando per il “giro” $\dots \rightarrow o \rightarrow \text{reset} \rightarrow i \rightarrow \dots$, dato che in N ogni nodo sta su un cammino $i \rightarrow o$. Questo è importante perché vale il teorema: un sistema weakly connected che è live e bounded è anche strongly connected; e reset garantisce proprio questa proprietà, rendendo “innocua” la trasformazione e abilitando il main theorem.

Da dire all'orale — short-circuit net

$N^\star = N + \text{transizione reset } (o \rightarrow i)$. Rende il processo ripetibile all'infinito, così liveness e boundedness diventano applicabili; inoltre rende la rete strongly connected. È il ponte per il main theorem: $N \text{ sound} \iff N^\star \text{ live e bounded}$.

1.4 Disegnare una rete che viola la proper completion

Domanda ×2

Disegna una rete che viola la proper completion.

Qui il professore vuole vedere se hai capito *davvero* la condizione 3. La proper completion dice: nell'istante in cui o riceve il token, la rete deve essere vuota ovunque. Per violarla basta costruire un workflow net in cui, quando il token arriva in o , resta ancora un token “dimenticato” da qualche altra parte.

Il modo più semplice e classico da disegnare alla lavagna è usare un AND-split iniziale i cui rami non vengono risincronizzati prima della fine. In parole:

- dal place iniziale i parte una transizione t_{start} che è un AND-split: produce un token sia in un place p_1 sia in un place p_2 (due rami paralleli);
- il ramo di p_1 prosegue con una transizione t_a che porta il token fino a o ;
- il ramo di p_2 , invece, termina in una transizione t_b (o resta fermo) senza mai ricongiungersi al place finale o .

Quando t_a scatta e mette il token in o , il ramo di p_2 ha ancora il suo token pendente: la marcatura raggiunta è “ $o + p_2$ ” (o simile), che ha $M(o) \geq 1$ ma $M \neq o$. Questo viola esattamente la condizione $M(o) \geq 1 \Rightarrow M = o$.

La lettura da dare all'orale: il difetto è un AND-split senza AND-join corrispondente — si aprono due fili di esecuzione paralleli ma se ne chiude solo uno, e l'altro lascia “spazzatura” nella rete a caso concluso. È il difetto tipico dei processi mal progettati con parallelismo sbilanciato.

Attenzione — coerenza col main theorem

Su questa rete, N^* risulterebbe unbounded: ad ogni giro reset riporta il token da o a i , ma il token residuo in p_2 si accumula, facendo crescere i token senza limite. È la controprova comportamentale della violazione di proper completion, coerente con “bounded $\Rightarrow$ proper completion”.

Capitolo 2

Invarianti

Gli invarianti sono l'apice degli strumenti algebrici del corso: trasformano domande sul *comportamento* (esplorare infiniti stati) in *calcoli di algebra lineare* sulla struttura della rete. Molto chiesti, soprattutto gli S-invariant.

2.1 S-invariant: definizione, matematica, due teoremi

Domanda ×3

Cos'è un S-invariant: definizione, scrittura matematica ($I \cdot N = 0$, con attenzione alla convenzione delle slide), cosa rappresenta, e i due teoremi (in particolare: S-invariant positive $\Rightarrow$ boundedness).

Partirei dall'intuizione, che è quella delle monete. Un S-invariant assegna a ogni place un peso (un "valore"), in modo tale che la somma pesata dei token nella rete non cambi mai, qualunque scatto avvenga. L'immagine è: i token sono monete di taglio diverso a seconda del place in cui si trovano; una transizione conserva il valore totale solo se "quello che entra vale quanto quello che esce". Se questo vale per ogni transizione, la somma pesata è costante durante tutta l'esecuzione.

Definizione — S-invariant

Un S-invariant (o place-invariant) di una rete $N = (P, T, F)$ è un vettore I di lunghezza $|P|$, a valori razionali, che soddisfa

$$I \cdot N = 0$$

dove N è la matrice di incidenza. È un sistema lineare omogeneo: la soluzione banale $I = 0$ c'è sempre, e ogni combinazione lineare di S-invariant è ancora un S-invariant (le soluzioni formano uno spazio vettoriale).

Sulla convenzione delle slide — che è il dettaglio su cui il professore insiste — va detto esplicitamente: qui I è un vettore *riga* e la matrice di incidenza N ha i *place sulle righe* e le *transizioni sulle colonne*. Quindi $I \cdot N = 0$ è un vettore di lunghezza $|T|$ tutto nullo: una equazione per transizione. Se si usasse la convenzione trasposta bisognerebbe scrivere $N^T I = 0$; è per questo che conviene enunciare anche la caratterizzazione "a occhio", che è indipendente dalla convenzione:

Definizione — S-invariant — caratterizzazione sul disegno

$I : P \rightarrow \mathbb{Q}$ è un S-invariant se e solo se, per ogni transizione t , il peso totale in ingresso eguaglia quello in uscita:

$$\sum_{p \in \bullet t} I(p) = \sum_{p \in t \bullet} I(p)$$

È esattamente la condizione "monete in = monete out", transizione per transizione.

A questo punto enuncerei la proprietà fondamentale, che è il motivo per cui gli S-invariant sono utili, con la sua dimostrazione (breve e richiesta spesso):

Teorema — Proprietà fondamentale degli S-invariant

Se I è un S-invariant, allora per ogni marcatura raggiungibile $M \in [M_0]$:

$$I \cdot M = I \cdot M_0$$

cioè la somma pesata dei token è costante durante tutta l'esecuzione.

La dimostrazione usa il marking equation lemma. Se M è raggiungibile, esiste σ con $M_0 \xrightarrow{\sigma} M$, e per la marking equation $M = M_0 + N \cdot \vec{\sigma}$. Moltiplicando a sinistra per I :

$$I \cdot M = I \cdot M_0 + I \cdot N \cdot \vec{\sigma} = I \cdot M_0 + 0 \cdot \vec{\sigma} = I \cdot M_0$$

perché $I \cdot N = 0$. ■

Prima dei due teoremi applicativi serve una classificazione per segno: un S-invariant è semi-positive se $I \geq 0$ e $I \neq 0$ (nessun peso negativo, almeno uno positivo), ed è positive se $I(p) > 0$ per ogni place (caso più forte). Il *support* $\langle I \rangle = \{p \mid I(p) > 0\}$ è l'insieme dei place con peso positivo; per un invariante positive è tutto P .

Teorema — 1) S-invariant positive $\Rightarrow$ boundedness

Se la rete ha un S-invariant positive, allora è bounded. Anzi, ogni place è limitato esplicitamente: da

$$I(p)M(p) \leq I \cdot M = I \cdot M_0$$

segue

$$M(p) \leq \frac{I \cdot M_0}{I(p)}$$

un limite indipendente dalla marcatura raggiungibile. Basta esibire un S-invariant positive per certificare la boundedness, senza esplorare gli stati.

Teorema — 2) S-invariant semi-positive per disprovare la liveness

Se la rete è live, allora per ogni S-invariant semi-positive I vale $I \cdot M_0 > 0$. Per contrapposizione: se troviamo un S-invariant semi-positive con $I \cdot M_0 = 0$, la rete non è live. (Se il valore iniziale è zero e i pesi sono ≥ 0 , i place del support restano vuoti per sempre, quindi le transizioni collegate sono dead.)

C'è anche un terzo uso, che vale la pena citare: poiché $I \cdot M = I \cdot M_0$ per ogni M raggiungibile, se una marcatura M ha $I \cdot M \neq I \cdot M_0$ per qualche S-invariant, allora M non è raggiungibile. È un test veloce di non-raggiungibilità che evita il problema completo (EXPSPACE-hard).

Attenzione — le implicazioni non si invertono

Sono condizioni sufficienti o necessarie, non equivalenze. *Nessun* S-invariant positive non implica unbounded (potrebbe esserlo o no); $I \cdot M_0 > 0$ non implica live; $I \cdot M = I \cdot M_0$ non implica M raggiungibile. Gli invarianti danno certezze solo “in un verso”.

Da dire all'orale — S-invariant

Peso sui place con somma pesata conservata: $I \cdot N = 0$, equivalentemente $\sum_t I(p) = \sum_t I(p)$. Proprietà fondamentale: $I \cdot M = I \cdot M_0$ (prova con marking equation). Usi: positive $\Rightarrow$ bounded ($M(p) \leq I \cdot M_0 / I(p)$); semi-positive con $I \cdot M_0 = 0 \Rightarrow$ non-live; $I \cdot M \neq I \cdot M_0 \Rightarrow M$ irraggiungibile.

2.2 La proprietà fondamentale, con dimostrazione

Domanda ×2

Enuncia e dimostra la proprietà fondamentale degli S-invariant ($I \cdot M = I \cdot M_0$ per ogni M raggiungibile).

Questa è la stessa dimostrazione riportata sopra, ma se viene chiesta isolata conviene esporla in modo autonomo, mettendo in evidenza che si appoggia *tutta* sul marking equation lemma.

Enuncio: se I è un S-invariant (cioè $I \cdot N = 0$), allora per ogni marcatura raggiungibile $M \in [M_0]$ vale $I \cdot M = I \cdot M_0$. In parole, la quantità $I \cdot M$ (la somma pesata dei token) è un *invariante del sistema*: non cambia mai, per quanti scatti si eseguono.

Dimostrazione. Sia M raggiungibile: esiste una firing sequence σ tale che $M_0 \xrightarrow{\sigma} M$. Il marking equation lemma ci dice che

$$M = M_0 + N \cdot \vec{\sigma}$$

dove $\vec{\sigma}$ è il Parikh vector di σ (il conteggio di quante volte scatta ciascuna transizione). Moltiplico entrambi i membri a sinistra per il vettore I :

$$I \cdot M = I \cdot M_0 + I \cdot N \cdot \vec{\sigma}.$$

Ma per ipotesi I è un S-invariant, quindi $I \cdot N = 0$, e il termine $I \cdot N \cdot \vec{\sigma} = 0 \cdot \vec{\sigma} = 0$ si annulla. Resta

$$I \cdot M = I \cdot M_0. \quad \blacksquare$$

Un commento che fa buona impressione: la prova mostra *perché* la definizione $I \cdot N = 0$ è quella giusta. Non è una condizione arbitraria: è precisamente ciò che serve per far sparire il contributo degli scatti nell'equazione della marcatura, indipendentemente da *quali* e *quanti* scatti siano avvenuti. È il Parikh vector che “dimentica l'ordine”, ma qui non serve nemmeno l'ordine: conta solo che I annulli la matrice.

Da dire all'orale — proprietà fondamentale

$I \cdot M = I \cdot M_0$ per ogni M raggiungibile. Prova in due righe: marking equation $M = M_0 + N\vec{\sigma}$, moltiplico per I , uso $I \cdot N = 0$, il termine di scatto sparisce. È la base di tutti gli usi degli S-invariant.

Capitolo 3

Liveness e boundedness

Sono le proprietà comportamentali di base, e i loro legami con gli S-invariant e con le classi speciali di reti tornano continuamente. Diverse domande chiedono anche di *disegnare* controesempi: è il modo con cui il professore verifica che hai capito le definizioni e non solo memorizzato.

3.1 Boundedness: definizione, notazione, legame con gli S-invariant

Domanda ×2

Cos'è la boundedness: definizione, notazione matematica, legame con gli S-invariant.

La boundedness risponde alla domanda: esiste un limite al numero di token che un place può contenere? Se sì, quel place non può “traboccare”; se no, il sistema può accumulare risorse senza fine — che nei processi reali è quasi sempre un bug (per esempio richieste che si accodano senza mai essere smaltite).

Definizione — k-bounded, safe, bounded

- Un place p è k-bounded se nessuna marcatura raggiungibile ha più di k token in p .
- Un place è safe se è 1-bounded (al più un token).
- Un place è bounded se è k-bounded per qualche k ; una rete è bounded se tutti i suoi place lo sono, unbounded altrimenti.

Formalmente, una rete è bounded se

$$\exists k \in \mathbb{N}. \forall M \in [M_0]. \forall p \in P. M(p) \leq k$$

Il legame teorico da citare per primo è con l'occurrence graph: una rete è bounded se e solo se il suo reachability graph è finito. Se è k-bounded, con n place ci sono al più $(k + 1)^n$ marcature, quindi un numero finito di nodi; viceversa, se il grafo è finito, il massimo numero di token in un place su tutti i nodi è un limite. Come corollario, una rete è unbounded sse il suo reachability graph è infinito (e allora si ricorre al coverability graph per rappresentarlo in modo finito con il simbolo ω).

Il legame con gli S-invariant, che è la parte specificamente richiesta, è il seguente: se la rete ha un S-invariant positive I (tutti i pesi > 0), allora è bounded. La ragione è la proprietà fondamentale $I \cdot M = I \cdot M_0$: poiché tutti i pesi sono positivi, per ogni place p

$$I(p)M(p) \leq I \cdot M = I \cdot M_0 \implies M(p) \leq \frac{I \cdot M_0}{I(p)}$$

che è un limite esplicito, uguale per tutte le marcature raggiungibili. Quindi esibire un S-invariant positive è un certificato di boundedness, ottenuto con puro calcolo algebrico senza esplorare gli stati. È esattamente il tipo di ragionamento “strutturale $\rightarrow$ comportamentale” che caratterizza gli invarianti.

Aggiungerei il legame duale con i T-invariant: un sistema *live* senza T-invariant positive è unbounded; equivalentemente, se una rete ha un S-invariant positive (quindi è bounded) ma nessun T-invariant positive, allora non può essere live.

Da dire all'orale — boundedness

$\exists k. \forall M \in [M_0]. \forall p. M(p) \leq k$. Bounded $\iff$ reachability graph finito. Legame con S-invariant: un S-invariant *positive* certifica la boundedness, con $M(p) \leq I \cdot M_0 / I(p)$.

3.2 Tutto su liveness

Domanda $\times 1$

Racconta tutto sulla liveness: definizione, notazione, place-live, non-live, e i legami con S-net, T-net e free-choice net.

La liveness riguarda il comportamento a lungo termine: una data attività potrà sempre, prima o poi, essere eseguita? Oppure può capitare di finire in uno stato da cui quell'attività non sarà mai più possibile? È la proprietà che garantisce che un processo non “muoia pezzo per pezzo”.

La cosa cruciale è che la liveness è *molto* più forte del semplice “la transizione scatta almeno una volta”. Una transizione può scattare all'inizio e poi morire per sempre: sarebbe *non-dead* ma non live.

Definizione — Transizione live

Una transizione t è live se da *ogni* marcatura raggiungibile si può *sempre* tornare a poterla abilitare:

$$\forall M \in [M_0]. \exists M' \in [M]. M' \xrightarrow{t}$$

Una rete è live se tutte le sue transizioni sono live.

Il cuore è la struttura “ $\forall M \exists M'$ ”: non importa dove sei finito, una via per riabilitare t c'è sempre. Le definizioni collegate si ottengono giocando con i quantificatori: t è dead se non è abilitata in nessuna marcatura raggiungibile ($\forall M. M \not\xrightarrow{t}$); è non-live (la negazione di live) se *esiste* una marcatura da cui, comunque si prosegua, t non torna mai abilitabile ($\exists M. \forall M' \in [M]. M' \not\xrightarrow{t}$); è non-dead se scatta almeno una volta. Il legame chiave: un sistema non è live sse ha una transizione che a un certo punto può *diventare* dead.

La stessa idea si trasporta ai place: un place p è live se da ogni marcatura raggiungibile si può sempre tornare ad averlo marcato ($\forall M. \exists M' \in [M]. M'(p) > 0$). Una rete è place-live se tutti i suoi place lo sono. Il legame tra le due nozioni: live $\Rightarrow$ place-live (in ogni rete), ma *non* viceversa in generale. La dimostrazione di “live $\Rightarrow$ place-live”: dato p e M , poiché non ci sono nodi isolati esiste una transizione t che tocca p ; per liveness da M si raggiunge uno scatto di t , e prima o dopo quello scatto p è marcato.

C'è poi una proprietà più debole, la deadlock-freedom: la rete non si blocca mai del tutto, cioè ogni marcatura raggiungibile abilita *qualche* transizione ($\forall M. \exists t. M \xrightarrow{t}$). Vale live $\Rightarrow$ deadlock-free (dimostrabile per assurdo: se ci fosse un deadlock M , l'unica marcatura raggiungibile da M è M stessa, contraddicendo la liveness di una qualsiasi t) e place-live $\Rightarrow$ deadlock-free, ma nessuna di queste implicazioni si inverte. In sintesi la gerarchia è: live $\Rightarrow$ place-live $\Rightarrow$ deadlock-free.

Sui legami con le classi di reti, che è la parte più ricca:

- S-system: è live $\iff$ è strongly connected e $M_0(P) \geq 1$ (c'è almeno un token in giro). Con un token e la strong connectedness, si sposta il token fino a qualunque transizione la si voglia abilitare.
- T-system: è live $\iff$ ogni circuito è marcato in M_0 ($M_0(\gamma) > 0$ per ogni circuito γ). Se un circuito ha zero token, resta vuoto per sempre e blocca le transizioni che vi si affacciano.
- Free-choice net: qui place-live $\iff$ live (l'implicazione debole si inverte, grazie alla proprietà free-choice); inoltre, per il teorema di Commoner, è live $\iff$ ogni proper siphon

include un trap inizialmente marcato. Decidere la sola liveness è NP-completo, ma liveness + boundedness è polinomiale (Rank Theorem).

Da dire all'orale — liveness

Live: $\forall M \exists M' \in [M]. M' \xrightarrow{t}$ (da ovunque si può *sempre* riabilitare t). Gerarchia: live $\Rightarrow$ place-live $\Rightarrow$ deadlock-free (nessuna si inverte in generale). Classi: S-system live $\iff$ strongly connected + token; T-system live $\iff$ ogni circuito marcato; free-choice: place-live $\iff$ live, e Commoner via siphon/trap.

3.3 Disegna una rete deadlock-free ma non live

Domanda $\times 1$

Disegna una rete deadlock-free ma non live.

Questa domanda serve a far vedere che deadlock-freedom $\nRightarrow$ live, cioè che l'implicazione va in un solo verso. Devo costruire una rete che gira per sempre (c'è sempre almeno una mossa disponibile) ma che ha una transizione che, a un certo punto, non scatterà mai più.

L'esempio canonico: due parti che “si contendono” l'evoluzione, con una transizione che rimane tagliata fuori. In pratica:

- un place p con un token che alimenta uno XOR-split: due transizioni t_1 e t_2 competono per quel token;
- se scatta t_2 , il token va in un place q da cui una transizione t_3 lo riporta in p (un ciclo $p \rightarrow t_2 \rightarrow q \rightarrow t_3 \rightarrow p$ che può girare all'infinito);
- t_1 , invece, porta il token in un ramo che a un certo punto non consente più di tornare in p — oppure, più semplicemente, la scelta ripetuta di t_2 fa sì che t_1 resti abilitata ma possa non scattare mai.

La chiave da spiegare: finché il sistema continua a scegliere il ciclo t_2 – t_3 , è deadlock-free (da ogni marcatura c'è sempre una transizione abilitata, quelle del ciclo). Ma se pensiamo a una transizione “laterale” che, dopo una certa scelta, non riceve più il suo token — per esempio una t_4 che dipende da un place che si svuota definitivamente in uno dei rami — allora quella transizione diventa dead da quella marcatura in poi, e la rete non è live.

Il controesempio più compatto che disegnerei alla lavagna è la classica rete con un place di scelta e un ramo “a senso unico”: il ciclo tiene viva la rete (no deadlock), ma la transizione del ramo abbandonato muore. All'orale basta enunciare chiaramente: “la rete gira per sempre grazie al ciclo, quindi è deadlock-free; ma esiste una marcatura da cui la transizione t non è più riabilitabile, quindi non è live”.

Attenzione — il refuso frequente

Nelle testimonianze compare spesso “draw a net deadlock-free *but* live”: è quasi certamente un refuso per “deadlock-free ma *non* live”. Una rete live è per definizione deadlock-free, quindi la domanda “deadlock-free e live” sarebbe banale. Vale la pena farlo notare al professore.

3.4 Disegna una rete non bounded

Domanda $\times 1$

Disegna una rete non bounded.

Una rete è unbounded se esiste un place in cui può comparire un numero arbitrariamente grande di token. Il modo più diretto per costruirne una è un ciclo che produce token in un place “pozzo” senza mai consumarli.

L'esempio minimale:

- un place i con un token, una transizione t_1 che lo porta in un place p ;
- da p una transizione t_2 che, oltre a far proseguire il flusso (rimettendo un token in p , o tornando indietro in un ciclo), deposita anche un token in un place o ;
- il place o non ha transizioni uscenti che ne consumino i token.

Ad ogni giro del ciclo, t_2 aggiunge un token in o senza che nessuno lo tolga: o accumula 1, 2, 3, ... token senza limite. Quindi o è unbounded e la rete è unbounded.

La lettura all'orale, che lega la costruzione alla teoria: il reachability graph di questa rete è infinito (una marcatura diversa per ogni valore di o), coerente con “unbounded $\iff$ reachability graph infinito”. Per rappresentarlo in modo finito si usa il coverability graph, che rileva il pattern $M \subsetneq M'$ lungo il cammino (la marcatura si ripete ma con un token in più in o) e marca o con il simbolo ω (“infiniti token”), collassando gli infiniti nodi in uno solo.

Da dire all'orale — rete unbounded

Ciclo che produce token in un place o senza consumarli: o accumula token all'infinito. Reachability graph infinito; il coverability graph lo domina segnando o con ω (rileva $M \subsetneq M'$ lungo il cammino).

Capitolo 4

Matrici e marking equation

L'approccio algebrico è la base di invarianti e soundness. La marking equation, in particolare, viene chiesta con la dimostrazione.

4.1 Marking equation lemma, con dimostrazione

Domanda ×2

Enuncia e dimostra il marking equation lemma.

Introdurrei l'idea: con la matrice di incidenza possiamo calcolare *dove* arriva la rete dopo una sequenza di scatti senza simularli uno per uno. Lo scatto di una singola transizione è una somma di vettori, $M' = M + \vec{t}$, dove $\vec{t}$ è la colonna di t nella matrice di incidenza (l'effetto netto di t su ogni place). Il marking equation lemma generalizza questo a una sequenza qualsiasi, usando il Parikh vector per contare gli scatti ignorando l'ordine.

Teorema — Marking equation lemma

Se $M \xrightarrow{\sigma} M'$, allora

$$M' = M + N \cdot \vec{\sigma}$$

dove N è la matrice di incidenza e $\vec{\sigma}$ il Parikh vector di σ (il conteggio delle occorrenze di ciascuna transizione in σ).

Dimostrazione per induzione sulla lunghezza di σ .

Caso base ($\sigma = \epsilon$, la sequenza vuota): il Parikh vector è $\vec{\sigma} = 0$, quindi $M + N \cdot 0 = M = M'$, e infatti la sequenza vuota non cambia la marcatura. Vero banalmente.

Passo induttivo ($\sigma = \sigma' t$, cioè σ è una sequenza più corta σ' seguita da un ultimo scatto t). Sia $M \xrightarrow{\sigma'} M'' \xrightarrow{t} M'$. Per l'ipotesi induttiva applicata a σ' vale $M'' = M + N \cdot \vec{\sigma}'$; e per il singolo scatto t vale $M' = M'' + N \cdot \vec{t}$. Sostituendo:

$$\begin{aligned} M' &= M'' + N \cdot \vec{t} \\ &= (M + N \cdot \vec{\sigma}') + N \cdot \vec{t} \\ &= M + N \cdot (\vec{\sigma}' + \vec{t}) \\ &= M + N \cdot \vec{\sigma} \end{aligned}$$

dove nell'ultimo passaggio ho usato la definizione ricorsiva del Parikh vector, $\vec{\sigma' t} = \vec{\sigma'} + \vec{t}$. ■

Vale la pena chiudere con le due conseguenze che rendono il lemma utile. La prima, sorprendente: la marcatura raggiunta dipende solo dal numero di occorrenze di ciascuna transizione, non dall'ordine — quindi ogni permutazione *eseguibile* delle stesse transizioni porta alla stessa marcatura finale. La seconda: il lemma è una condizione necessaria, non sufficiente. Il calcolo $M + N \cdot \vec{\sigma}$ si può fare per qualsiasi $\vec{\sigma}$; se il risultato ha una componente negativa, la sequenza *non* è eseguibile (non si può avere un numero negativo di token). Un valore ≥ 0 ovunque, invece, non garantisce l'eseguibilità: è solo un test di *non-eseguibilità*.

Da dire all'orale — marking equation

$M \xrightarrow{\sigma} M' \Rightarrow M' = M + N\vec{\sigma}$. Prova per induzione sulla lunghezza di σ (base: $\vec{\epsilon} = 0$; passo: $\vec{\sigma}'t = \vec{\sigma}' + \vec{t}$). Conseguenze: il risultato non dipende dall'ordine; una componente negativa certifica la non-eseguibilità. È il motore di S-invariant e monotonicity.

4.2 Matrice di incidenza

Domanda ×1

Cos'è la matrice di incidenza: cosa rappresentano righe e colonne, quali sono i valori possibili.

L'osservazione chiave da cui parte tutto: la *variazione* del numero di token in un place p causata dallo scatto di una transizione t non dipende dalla marcatura corrente, ma solo dalla struttura della rete (dalla flow relation). Quindi possiamo pre-calcolare, una volta per tutte, l'effetto di ogni transizione su ogni place — ed è proprio la matrice di incidenza.

Definizione — Matrice di incidenza

Data $N = (P, T, F)$, la matrice di incidenza $N : (P \times T) \rightarrow \{-1, 0, +1\}$ è

$$N(p, t) = \begin{cases} -1 & \text{se } (p, t) \in F \wedge (t, p) \notin F \quad (p \text{ solo input di } t) \\ +1 & \text{se } (p, t) \notin F \wedge (t, p) \in F \quad (p \text{ solo output di } t) \\ 0 & \text{altrimenti (scollegati oppure self-loop)} \end{cases}$$

Ha $n = |P|$ righe (una per place) e $m = |T|$ colonne (una per transizione).

Rispondendo direttamente: le righe sono i place, le colonne sono le transizioni. Ogni colonna $\vec{t}_j$ è l'effetto della transizione t_j su tutti i place (quanti token toglie/aggiunge in ciascuno); ogni riga descrive come tutte le transizioni influenzano un singolo place. I valori possibili sono solo tre: -1 (la transizione consuma un token da quel place), $+1$ (lo produce), 0 (nessun effetto netto).

Il punto delicato da menzionare è il caso 0 del self-loop: se p è sia input sia output di t , la transizione consuma e ri-produrre un token, quindi la variazione netta è zero — e la matrice *non distingue* un self-loop da “nessun collegamento”. Questa perdita di informazione è la ragione per cui l'approccio algebrico, pur potentissimo, non cattura *tutto* il comportamento della rete.

Da dire all'orale — matrice di incidenza

Righe = place, colonne = transizioni. Valori $\{-1, 0, +1\}$: -1 consuma, $+1$ produce, 0 nessun effetto netto o self-loop. La colonna di t è il vettore $\vec{t}$ usato nella marking equation. Attenzione: perde i self-loop.

4.3 Parikh vector

Domanda ×1

Cos'è il Parikh vector?

Il Parikh vector serve a gestire una *sequenza* di scatti sommandone gli effetti in un colpo solo. L'idea è contare quante volte scatta ciascuna transizione, dimenticando l'ordine.

Definizione — Parikh vector

Data una sequenza $\sigma \in T^*$, il suo Parikh vector $\vec{\sigma} : T \rightarrow \mathbb{N}$ associa a ogni transizione t il numero di occorrenze di t in σ . Per una singola transizione, $\vec{t}$ è il vettore con un 1 nella posizione di t e 0 altrove. Definizione ricorsiva:

$$\vec{\epsilon} = 0, \quad \vec{\sigma t} = \vec{\sigma} + \vec{t}$$

Un esempio chiarisce: se $\sigma = t_3 t_5 t_3 t_4 t_2$, allora $\vec{\sigma} = [0, 1, 2, 1, 1]$ (zero t_1 , un t_2 , due t_3 , un t_4 , un t_5).

Il punto essenziale da sottolineare è che il Parikh vector dimentica l'ordine: due sequenze con gli stessi scatti in ordine diverso hanno lo stesso Parikh vector. Questo è esattamente ciò che serve nella marking equation, perché — come dice il lemma — la marcatura finale dipende solo da *quante* volte scatta ciascuna transizione, non da *in che ordine*. Il Parikh vector è anche l'oggetto su cui si definiscono i T-invariant: $\vec{\sigma}$ è un T-invariant sse la sequenza σ riporta la rete alla marcatura di partenza.

Da dire all'orale — Parikh vector

Conta le occorrenze di ogni transizione in σ , ignorando l'ordine. $\vec{\epsilon} = 0$, $\vec{\sigma t} = \vec{\sigma} + \vec{t}$. È l'input della marking equation ($M' = M + N\vec{\sigma}$) e la base dei T-invariant.

Capitolo 5

Free-choice net

Le free-choice net sono la classe “giusta” per i processi reali: abbastanza generali da modellarli, abbastanza strutturate da rendere la soundness decidibile in tempo polinomiale. Il Rank Theorem e il legame con siphon/trap sono le domande tipiche.

5.1 Rank theorem

Domanda ×2

Enuncia il Rank Theorem: tutte le condizioni, più l’algoritmo per verificare il punto sul rank.

Contestualizzerei subito il risultato con il “contrasto” che è il take-home della lezione. Decidere la sola liveness di una free-choice net è NP-completo (riduzione da SAT). Ma decidere liveness E boundedness insieme — che è esattamente ciò che serve per la soundness, dato che N sound $\iff N^*$ live e bounded — diventa polinomiale. Aggiungere il vincolo di boundedness *semplifica* il problema invece di complicarlo: è il Rank Theorem che lo garantisce.

Prima serve la nozione di cluster. Il cluster di un nodo x , scritto $[x]$, è il più piccolo insieme tale che: $x \in [x]$; se un place $p \in [x]$ allora tutte le sue transizioni uscenti $p \bullet \subseteq [x]$; se una transizione $t \in [x]$ allora tutti i suoi place entranti $\bullet t \subseteq [x]$. Si “chiude” alternando place $\rightarrow$ transizioni uscenti e transizione $\rightarrow$ place entranti. I cluster partizionano i nodi; in una free-choice un cluster è una “zona di scelta libera”. C_N è l’insieme dei cluster.

Teorema — Rank Theorem

Un free-choice system (P, T, F, M_0) è live e bounded $\iff$ valgono tutte:

1. ha almeno un place e una transizione;
2. è connesso;
3. M_0 marca ogni proper siphon;
4. ha un S-invariant positivo (tutte le componenti > 0);
5. ha un T-invariant positivo;
6. $\text{rank}(N) = |C_N| - 1$: il rango della matrice di incidenza è il numero di cluster meno uno.

Il punto forte da comunicare è che ognuna delle sei condizioni si verifica in tempo polinomiale: connessione (visita del grafo), invarianti positivi e rango (algebra lineare), tutto poly. Quindi la soundness dei workflow net free-choice è decidibile in tempo polinomiale — la giustificazione teorica del perché tool come WoPeD funzionano bene sui processi reali, che sono quasi sempre free-choice.

Sull’algoritmo per la condizione sul rank e su quella dei siphon (che il professore chiede in coppia): il rango della matrice di incidenza si calcola con la normale eliminazione di Gauss, in tempo polinomiale; i cluster si trovano con una chiusura sul grafo, altrettanto poly; poi si confronta $\text{rank}(N)$ con $|C_N| - 1$. Per la condizione 3 (“ M_0 marca ogni proper siphon”) si usa l’algoritmo del massimo siphon non marcato, che descrivo nella prossima sezione: si parte dai place vuoti e si rimuovono iterativamente quelli che violano la condizione di siphon; se alla fine non resta nulla, allora ogni proper siphon è marcato.

Da dire all'orale — Rank Theorem

Free-choice live+bounded $\iff$ (1) non vuoto, (2) connesso, (3) M_0 marca ogni proper siphon, (4) S-invariant positivo, (5) T-invariant positivo, (6) $\text{rank}(N) = |C_N| - 1$. Tutte poly $\Rightarrow$ soundness dei WF net free-choice in tempo polinomiale. Contrasto: la sola liveness è NP-completa.

5.2 Free-choice net e teorema di Commoner

Domanda $\times 1$

Racconta tutto sulle free-choice net: definizione e Commoner's theorem (siphon/trap).

La motivazione è eliminare l'interferenza fra conflitto e sincronizzazione. In una rete qualsiasi, due transizioni possono passare dall'essere indipendenti all'essere in conflitto a seconda di cosa scatta prima nel resto del sistema — e questo rende la scelta “non libera”. Le free-choice impediscono che una scelta fra transizioni sia influenzata da altro.

Definizione — Free-choice net

Un Petri net è free-choice se, per ogni coppia di transizioni, i pre-set sono disgiunti oppure uguali:

$$\forall t, t' \in T. \quad \bullet t \cap \bullet t' = \emptyset \vee \bullet t = \bullet t'$$

(equivalentemente, per ogni coppia di place i post-set sono disgiunti o uguali).

L'intuizione: se due transizioni condividono anche un solo place di input, allora devono dividerli *tutti*. Così sono sempre abilitate insieme e la scelta fra loro è davvero libera, dipende solo dai token nei place comuni. Da questo segue la proprietà fondamentale: se M abilita t e $t \in p\bullet$, allora M abilita ogni altra $t' \in p\bullet$ (o sono tutte abilitate, o nessuna). Da citare anche che S-net e T-net sono casi particolari di free-choice (nell'S-net ogni $\bullet t$ è un singoletto, nel T-net ogni $p\bullet$ lo è), quindi le free-choice sono una loro super-classe.

Per la liveness servono siphon e trap, due strutture duali sui place:

Definizione — Siphon e trap

- Un insieme di place R è un siphon se $\bullet R \subseteq R\bullet$: ogni transizione che *produce* in R richiede anche un place di R come input. Conseguenza: se R è vuoto, resta vuoto per sempre.
- R è un trap se $R\bullet \subseteq \bullet R$: ogni transizione che *consuma* da R produce anche in R . Conseguenza: se R è marcato, resta marcato per sempre.

R è *proper* se $R \neq \emptyset$.

In *ogni* rete vale che, se il sistema è live, ogni proper siphon è marcato (contronominale: un proper siphon non marcato $\Rightarrow$ non live). Per le free-choice questa condizione diventa esatta grazie ai trap:

Teorema — Teorema di Commoner (free-choice)

Un free-choice system è live $\iff$ ogni proper siphon include un trap inizialmente marcato (equivalentemente, il trap massimale di ogni proper siphon è marcato).

Contronominale: è non-live $\iff$ esiste un proper siphon il cui trap massimale è non marcato.

L'idea: un siphon non marcato è una condanna (resta vuoto e blocca le transizioni), *a meno che* contenga un trap marcato che gli inietta token per sempre. La caratterizzazione via trap massimale è comoda perché il trap massimale è unico (i trap sono chiusi per unione) e calcolabile. Come detto, decidere questa condizione (la sola liveness) è NP-completo, ma con la boundedness in più si torna

al Rank Theorem, polinomiale.

Da dire all'orale — free-choice + Commoner

Free-choice: $\bullet t \cap \bullet t' = \emptyset$ o $\bullet t = \bullet t'$ (scelta libera). Siphon ($\bullet R \subseteq R \bullet$): vuoto resta vuoto. Trap ($R \bullet \subseteq \bullet R$): marcato resta marcato. Commoner: live $\iff$ ogni proper siphon include un trap marcato in M_0 .

5.3 Algoritmo per il massimo siphon non marcato

Domanda $\times 1$

Descrivi l'algoritmo per trovare il massimo siphon non marcato.

Questo algoritmo serve a verificare la condizione 3 del Rank Theorem (“ M_0 marca ogni proper siphon”) in tempo polinomiale, senza enumerare tutti i $2^{|P|}$ sottoinsiemi di place. L'idea è partire dall'insieme più grande possibile di place “sospetti” e togliere iterativamente quelli che non possono far parte di un siphon non marcato.

Il ragionamento: cerchiamo il *più grande* insieme R di place che sia (a) un siphon e (b) completamente non marcato in M_0 . Si procede così:

1. Inizializza R con tutti i place vuoti in M_0 (quelli con $M_0(p) = 0$): un siphon non marcato può contenere solo place vuoti, quindi partiamo dal candidato massimale.
2. Controlla la condizione di siphon $\bullet R \subseteq R \bullet$. In pratica: per ogni transizione t che *produce* in R (cioè $t \in \bullet R$), verifica che t *consumi* anche da R (cioè $t \in R \bullet$). Se una transizione produce in un place $p \in R$ ma non consuma da nessun place di R , allora p potrebbe ricevere token dall'esterno: p non può stare nel siphon non marcato, e lo rimuoviamo da R .
3. Ripeti il passo 2 finché non ci sono più place da rimuovere (la rimozione di un place può far “saltare” la condizione per altri, quindi si itera fino a stabilizzazione).
4. Risultato: l'insieme R rimasto è il massimo siphon non marcato. Se R è vuoto, allora *ogni* proper siphon è marcato (la condizione 3 del Rank Theorem è soddisfatta). Se $R \neq \emptyset$, abbiamo trovato un proper siphon non marcato, e quindi (in una free-choice, se non contiene un trap marcato) un certificato di non-liveness / non-soundness.

Il punto sulla complessità: ogni iterazione rimuove almeno un place o termina, quindi si fanno al più $|P|$ iterazioni, ciascuna con un controllo polinomiale sulle transizioni. L'intero algoritmo è polinomiale — ed è proprio questo che permette al Rank Theorem di decidere liveness + boundedness (e quindi la soundness) in tempo polinomiale.

Da dire all'orale — massimo siphon non marcato

Parti da $R =$ tutti i place vuoti; toglì iterativamente ogni $p \in R$ in cui una transizione produce senza consumare da R (viola $\bullet R \subseteq R \bullet$); itera fino a stabilizzazione. Se $R = \emptyset$ ogni proper siphon è marcato; altrimenti R è un siphon non marcato (potenziale non-liveness). Poly.

Capitolo 6

S-system e T-system

Sono le due classi “pulite” e duali: gli S-system vietano la sincronizzazione, i T-system vietano la scelta. Per entrambe, le proprietà comportamentali diventano criteri strutturali verificabili a occhio.

6.1 Proprietà fondamentale degli S-system

Domanda ×1

Cos'è un S-net e qual è la proprietà fondamentale degli S-system (conservazione dei token)?

Definirei prima la classe. Una rete è un S-net se ogni transizione ha esattamente un input place e un output place: $\forall t. |\bullet t| = 1 = |t \bullet|$. La “S” viene da *Stellen*, “place” in tedesco. Una transizione così non può sincronizzare (non aspetta più input) né spezzare (non produce più output): nell'S-net la sincronizzazione è *vietata*. Un S-system è semplicemente un S-net con una marcatura iniziale.

Da questa struttura discende immediatamente una legge di conservazione:

Teorema — Proprietà fondamentale degli S-system

Il numero totale di token è invariante. Detto $M(P) = \sum_{p \in P} M(p)$ il conteggio totale, per ogni marcatura raggiungibile $M \in [M_0]$:

$$M(P) = M_0(P)$$

La dimostrazione è una riga: ogni scatto di una transizione t toglie esattamente un token (dall'unico input place) e ne mette esattamente uno (nell'unico output place), quindi

$$M'(P) = M(P) - |\bullet t| + |t \bullet| = M(P) - 1 + 1 = M(P).$$

Il totale non cambia mai. È l'analogo di una legge di conservazione della fisica: il token “si sposta” ma non si crea né si distrugge.

Le conseguenze immediate, tutte da citare:

- Sempre bounded: da $M(p) \leq M(P) = M_0(P)$, nessun place può superare $M_0(P)$ token. Un S-system è k -bounded con $k = M_0(P)$.
- Test di raggiungibilità gratis: se $M(P) \neq M_0(P)$, allora M non è raggiungibile (se parto con 6 token totali, una marcatura con 7 è impossibile).
- S-invariant uniformi: gli unici S-invariant di un S-net connesso sono i vettori costanti $I = [k, k, \dots, k]$, coerente con “ogni token vale uguale, la somma si conserva”.

Se c'è tempo, aggiungerei la caratterizzazione della liveness, che è puramente strutturale: un S-system è live $\iff$ è strongly connected e $M_0(P) \geq 1$ (c'è almeno un token). Con un token in giro e la strong connectedness, per abilitare qualunque transizione basta spostare il token fino a lei lungo un cammino. E il take-home sui workflow net: ogni workflow net che sia un S-net è *safe e sound* — un solo caso che scorre, nessuna sincronizzazione, correttezza automatica.

Da dire all'orale — S-system

S-net: ogni transizione 1-in-1-out (vieta la sincronizzazione). Proprietà fondamentale: il totale $M(P) = M_0(P)$ è conservato (uno scatto toglie 1 e mette 1). Conseguenze: sempre bounded, raggiungibilità = conteggio token, S-invariant uniformi. WF net S-net $\Rightarrow$ safe e sound.

6.2 Boundedness e liveness degli S-system

Domanda $\times 1$

Cosa si può dire su boundedness e liveness degli S-system?

Questa domanda è il completamento naturale della precedente, e conviene rispondere separando nettamente le due proprietà, perché per gli S-system entrambe hanno una risposta strutturale semplice.

Sulla boundedness: un S-system è sempre bounded, senza alcuna ipotesi aggiuntiva. La ragione è la conservazione del numero totale di token: poiché $M(P) = M_0(P)$ per ogni marcatura raggiungibile, e ogni place ha al più tutti i token, vale $M(p) \leq M(P) = M_0(P)$ per ogni place p . Quindi il sistema è k -bounded con $k = M_0(P)$: il numero di token iniziale è un tetto invalicabile. Non serve costruire alcun reachability graph, la boundedness è garantita dalla struttura.

Sulla liveness, invece, serve una condizione, ed è la caratterizzazione strutturale elegante:

Teorema — Liveness degli S-system

Un S-system (N, M_0) è live $\iff$

$$N \text{ è strongly connected} \quad \text{e} \quad M_0(P) \geq 1$$

L'intuizione della direzione " $\Leftarrow$ ": con almeno un token in circolazione e la strong connectedness, per abilitare una qualsiasi transizione t basta spostare il token fino a lei lungo un cammino orientato (che esiste, per la strong connectedness). Poiché ogni transizione dell'S-net non fa altro che spostare l'unico token, si può sempre portarlo dove serve. Se invece la rete non è strongly connected, c'è una parte da cui il token può "uscire" senza tornare, e le transizioni lì diventano dead; se $M_0(P) = 0$ non c'è alcun token e nulla scatta.

Combinando le due si ottiene anche una caratterizzazione esatta della raggiungibilità — cosa rara, dato che in generale è EXPSPACE-hard: in un S-system *live* (quindi strongly connected), una marcatura M è raggiungibile $\iff M(P) = M_0(P)$. Basta contare i token: se il totale coincide, la marcatura si raggiunge spostando i token uno per volta nei place giusti.

Il pagamento finale, da dire come conclusione: mettendo insieme "sempre bounded" e "live sse strongly connected + token", si dimostra che ogni workflow net che è un S-net è safe e sound. La catena: $N^\star$ resta un S-net (quindi bounded); è strongly connected per via del reset e ha $M_0(P) = 1$ (quindi live); live + bounded $\iff N$ sound. Il singolo token che scorre garantisce automaticamente la correttezza.

Da dire all'orale — boundedness e liveness S-system

Boundedness: *sempre* bounded (k -bounded con $k = M_0(P)$, per conservazione). Liveness: live $\iff$ strongly connected e $M_0(P) \geq 1$. Insieme: WF net S-net $\Rightarrow$ safe e sound (via $N^\star$).

S-coverability

Serve a certificare “a pezzi” l’S-invariant positivo richiesto dal Rank Theorem, e a diagnosticare perché un net non è sound.

7.1 S-coverability: definizione e relazioni

Domanda ×1

Cos’è la S-coverability: definizione e relazione con S-net, liveness e boundedness.

Motiverei la nozione partendo dal Rank Theorem: una delle sei condizioni per “live e bounded” è “avere un S-invariant positivo”. Il problema pratico è *come* trovarlo in modo intuitivo, non solo risolvendo un sistema lineare. L’idea è che un caso reale è spesso composto da filoni di controllo paralleli (thread), ciascuno dei quali impone un ordine sui propri task. Se decomponiamo la rete in sotto-reti che sono S-net, ciascuna induce un S-invariant uniforme, e sommandoli si ottiene un S-invariant positivo sull’intera rete.

Per formalizzare serve prima la nozione di S-component, che è una sottorete “ben fatta”:

Definizione — S-component

Una subnet N' indotta da un insieme di nodi X è un S-component se:

1. è un S-net strongly connected (ogni transizione ha esattamente un place in ingresso e uno in uscita);
2. per ogni place $p \in X$, tutte le transizioni collegate a p (sia $\bullet p$ sia $p \bullet$) sono in X — “se prendi un place, prendi anche tutte le sue transizioni”.

La condizione 2 è cruciale: evita di “tagliare a metà” le scelte di un place, garantendo che la subnet catturi fedelmente il comportamento locale di quel filone. Un singolo S-component copre solo alcuni place; per coprirli tutti serve una collezione:

Definizione — S-cover e S-coverability

Un S-cover di N è un insieme C di S-component tale che ogni place di N appartenga ad almeno uno di essi. N è S-coverable se ammette un S-cover.

La ricetta pratica: ogni S-component induce un S-invariant *uniforme* (peso 1 sui place che copre, 0 altrove — è il risultato sugli S-net); sommando gli invarianti di tutti gli S-component di un S-cover, poiché ogni place è coperto da almeno uno, si ottiene un vettore con *tutte* le componenti positive, cioè un S-invariant positivo su tutta la rete. Ecco come si certifica “a pezzi” uno dei sei ingredienti del Rank Theorem.

La relazione con liveness e boundedness è data da un teorema (nella direzione contronominale diventa un test di non-soundness):

Teorema — S-coverability theorem

Se un free-choice system è live e bounded, allora è S-coverable.

Contronominale (test pratico):

$$\text{free-choice} \wedge \neg \text{S-coverable} \Rightarrow \neg(\text{live} \wedge \text{bounded})$$

in particolare, se N è free-choice e $N^\star$ non è S-coverable, allora N non è sound.

Il modo giusto di riassumere le relazioni all'orale: la S-coverability è la nozione “strutturale a filoni” che sta dietro all'S-invariant positivo; ogni S-component è un S-net (strongly connected, con il suo invariante uniforme); e per le free-choice “live e bounded” (cioè la soundness di $N^\star$) implica la S-coverability. Quindi la mancata S-coverability è un certificato — visivo e diagnosticabile — di non-soundness: dice *quali* place non riescono a stare in nessun filone S-net coerente, indicando dove il processo è mal strutturato.

Da dire all'orale — S-coverability

S-component: subnet S-net strongly connected che, per ogni suo place, include tutte le transizioni collegate. S-cover: insieme di S-component che copre ogni place. Ogni S-component dà un S-invariant uniforme; la loro somma è un S-invariant positivo. Teorema: free-choice live+bounded $\Rightarrow$ S-coverable; contronominale $\Rightarrow$ test di non-soundness.

Grafi di raggiungibilità

8.1 Reachability graph vs coverability graph

Domanda ×1

Qual è la differenza tra reachability graph e coverability graph?

Partirei da cosa sono entrambi e dal problema che il secondo risolve. Il reachability graph (o occurrence graph) è la rappresentazione esatta dello spazio degli stati: i nodi sono le marcature raggiungibili da M_0 , e c'è un arco etichettato t da M a M' ogni volta che $M \xrightarrow{t} M'$. Rappresenta *tutti e soli* i comportamenti reali della rete.

Il limite del reachability graph è che può essere infinito. Vale infatti il teorema: una rete è bounded $\iff$ il suo reachability graph è finito. Quindi, se la rete è unbounded, il reachability graph ha infiniti nodi (un place che accumula token dà una marcatura diversa per ogni valore) e non lo possiamo costruire né esplorare: l'algoritmo che lo genera non terminerebbe.

Il coverability graph è la soluzione a questo problema: è una sovra-approssimazione finita del reachability graph, che ammette marcature "con infiniti token" in un place, rappresentati dal simbolo ω (0∞).

Definizione — Coverability graph

Una sovra-approssimazione finita del reachability graph, i cui nodi sono extended bag $B : P \rightarrow \mathbb{N} \cup \{\omega\}$, dove ω marca un place illimitato. Le operazioni trattano ω come assorbente: $\omega + n = \omega$, $\omega - n = \omega$, $\omega \geq n$ per ogni n finito.

Il meccanismo con cui scopre i place unbounded sfrutta la monotonicità dei Petri net: se lungo il calcolo si trova $M_0 \xrightarrow{*} M \xrightarrow{*} M'$ con $M \subsetneq M'$ (la seconda contiene strettamente la prima), allora il "surplus" $L = M' - M$ può essere ripetuto all'infinito ($M, M + L, M + 2L, \dots$), quindi ogni place p con $L(p) > 0$ è unbounded e lo si marca con ω . È così che gli infiniti nodi (" $o, 2o, 3o, \dots$ ") collasano in un solo nodo " ωo ", mantenendo il grafo finito.

Le differenze da enunciare in modo netto:

- Il reachability graph è esatto ma eventualmente infinito; il coverability graph è sempre finito ma è una sovra-approssimazione.
- Il coverability graph non è unico: dipende dall'ordine con cui l'algoritmo sceglie marcature e transizioni. Il reachability graph è unico.
- Ogni firing sequence reale ha un cammino corrispondente nel coverability graph, ma non vale il viceversa: il coverability graph può suggerire comportamenti che nella rete reale non avvengono (è il prezzo dell'astrazione con ω). Solo i cammini che visitano marcature *finita* corrispondono sempre a vere firing sequence.
- Se il reachability graph è finito (rete bounded), allora coincide col coverability graph.

La sintesi operativa: si usa il reachability graph quando la rete è bounded (allora è finito ed esatto); si ricorre al coverability graph solo quando serve dominare l'infinito di una rete unbounded, accettando che sia una sovra-approssimazione. Tool come WoPeD calcolano un coverability graph proprio per essere sempre in grado di terminare.

Da dire all'orale — reachability vs coverability

Reachability graph: esatto, unico, ma infinito se la rete è unbounded (bounded $\iff$ finito).
Coverability graph: sempre finito grazie a ω (place illimitati), ma sovra-approssimazione e non unico. Coincidono se la rete è bounded. ω scoperto via monotonicità sul pattern $M \subsetneq M'$.

EPC vs BPMN

9.1 Differenza tra BPMN ed EPC, con la notazione

Domanda ×2

Qual è la differenza tra BPMN ed EPC? Disegnane la notazione.

Imposterei la risposta dicendo che EPC e BPMN sono due notazioni grafiche high-level per gli stessi processi, con elementi in corrispondenza abbastanza diretta; le differenze vere sono nella ricchezza della notazione e — soprattutto — nella capacità di modellare la comunicazione tra partecipanti.

Comincerei dagli ingredienti dell'EPC (Event-driven Process Chain), disegnandoli:

- Event — un esagono: qualcosa che è accaduto (pre- o post-condizione di una funzione).
- Function — un rettangolo arrotondato: un'attività, un'unità di lavoro.
- Connettori logici — cerchi contenenti $\wedge$ (AND), $\vee$ (OR) o XOR: gestiscono split e join del flusso.
- Control flow — una freccia tratteggiata che collega gli elementi.

Una regola caratteristica dell'EPC è l'alternanza: un evento è seguito da una funzione e viceversa (eventi e funzioni si alternano lungo il control flow).

Poi gli elementi BPMN (Business Process Model and Notation), disegnandone la corrispondenza:

- Event — un cerchio: il bordo ne dà il tipo (start = bordo sottile, intermediate = doppio bordo, end = bordo spesso), un marker interno il sotto-tipo (message, timer, error).
- Activity — un rettangolo arrotondato: un task (atomico) o un sub-process (composto). Corrisponde alla function EPC.
- Gateway — un rombo (diamante) con un marker interno: + per il parallelo (AND), $\times$ per l'esclusivo (XOR), $\circ$ per l'inclusivo (OR). Corrisponde al connettore EPC.
- Sequence flow — una freccia piena (l'ordine di esecuzione *dentro* un pool).
- Swimlane (pool e lane) — rettangoli che rappresentano i partecipanti/ruoli.
- Message flow — una freccia tratteggiata con un pallino, per la comunicazione *tra* pool diversi.

La corrispondenza si riassume bene in una tabella:

Concetto	EPC	BPMN
Evento	esagono	cerchio (bordo = tipo)
Attività	function (rett. arrotondato)	activity / task (rett. arrotondato)
Split / join logico	connettore (cerchio $\wedge$ $\vee$ XOR)	gateway (rombo + $\times$ $\circ$)
Flusso di controllo	control flow (freccia tratteggiata)	sequence flow (freccia piena)
Responsabilità	organization unit	swimlane (pool / lane)
Comunicazione	— (non nativa)	message flow (tra pool)

Le differenze sostanziali da evidenziare all'orale:

1. Le swimlane e il message flow. È la differenza più importante: BPMN ha nativamente i pool (partecipanti) collegati da message flow, quindi modella direttamente le collaboration (più organizzazioni che si scambiano messaggi). L'EPC non ha un costrutto nativo altrettanto forte per la comunicazione inter-organizzativa.
2. Alternanza evento/funzione. L'EPC *impone* l'alternanza tra eventi e funzioni; BPMN è più libero (gli eventi sono opzionali, spesso si mettono solo start ed end).
3. Ricchezza e complessità. BPMN è lo standard industriale de facto (nasce per ridurre la frammentazione di BPEL, BPML, UML AD, ...) con oltre 100 elementi grafici e una specifica di 500 pagine: molto espressivo ma difficile da imparare a fondo, e vendor diversi ne implementano sottoinsiemi diversi. L'EPC è più semplice e "accademico".
4. Il problema comune degli OR-join. Entrambi soffrono della semantica non-locale dell'OR-join/gateway inclusivo: decidere se aspettare o meno un secondo input richiede di sapere "dove i token non arriveranno mai", informazione non-locale. In entrambe le notazioni la raccomandazione pratica è evitare gli OR quando possibile.

Chiuderei con il collegamento all'analisi: entrambe si traducono in Petri net per verificarne la soundness — un BPMN *process diagram* in un workflow net, un BPMN *collaboration diagram* in un workflow system (più net che comunicano) — quindi la scelta della notazione non cambia gli strumenti di verifica, cambia solo l'espressività e la leggibilità del modello di partenza.

Da dire all'orale — EPC vs BPMN

Corrispondenza diretta: evento(esagono/cerchio), function/activity(rett. arrotond.), connettore/-gateway(cerchio/rombo), control/sequence flow. Differenza chiave: BPMN ha swimlane + message flow ⇒ modella le collaboration; EPC no. EPC impone l'alternanza evento/funzione; BPMN è più ricco (100+ simboli) ma complesso. Entrambi soffrono l'OR-join non-locale.

Process mining e conformance

Il process mining ricava un modello dai dati (log); la conformance misura quanto un modello e un log reale concordano. Le domande tipiche sono i passi dell' α -algorithm e la footprint matrix, che è il concetto-ponte tra i due.

10.1 Footprint matrix

Domanda ×2

Cos'è la footprint matrix / la notazione footprint?

Introdurrei l'oggetto partendo dalle relazioni d'ordine basate sul log, perché la footprint matrix non è altro che il modo compatto di raccogliere tutte. Dato un event log (un multiset di tracce, cioè sequenze di attività), la relazione di base è il directly-follows:

$a >_L b$ sse esiste una traccia in cui a è immediatamente seguito da b .

Da questa si derivano tre relazioni che descrivono la “struttura” tra due attività:

Definizione — Le relazioni d'ordine

- Causality $a \rightarrow_L b$: vale $a >_L b$ ma non $b >_L a$ (l'ordine è sempre lo stesso $\Rightarrow$ dipendenza causale, prima a poi b).
- Mutual exclusion $a \#_L b$: né $a >_L b$ né $b >_L a$ (le due attività non sono mai adiacenti).
- Concurrency $a \parallel_L b$: valgono *entrambi* $a >_L b$ e $b >_L a$ (si osservano entrambi gli ordini $\Rightarrow$ le attività sono in parallelo, indipendenti).

Definizione — Footprint matrix

La footprint matrix è una tabella con una riga e una colonna per ogni attività del log, dove ogni cella (a, b) riporta la relazione che intercorre tra a e b : una fra $\rightarrow$, $\leftarrow$, $\#$ o $\parallel$. È il “riassunto strutturale” completo del comportamento osservato.

Da dire sulla struttura della matrice: è antisimmetrica per le relazioni causali (se in (a, b) c'è $\rightarrow$, allora in (b, a) c'è $\leftarrow$), mentre $\#$ e $\parallel$ sono simmetriche (compaiono uguali nelle due celle speculari). Un log si dice completo rispetto al footprint se ogni coppia di attività che *può* susseguirsi lo fa almeno una volta nel log.

Il motivo per cui la footprint è importante è che serve in due punti del corso. Nella discovery (α -algorithm) è il punto di partenza per costruire la rete. Nella conformance è un metodo di confronto leggero: si calcola la footprint sia dal log sia dal modello (estraendo tracce dal net con la tecnica del *play-out*), poi si confrontano le due matrici. La metrica è semplicissima:

$$\text{conformance} = 1 - \frac{d}{n}$$

dove n è il numero totale di celle e d il numero di celle nella stessa posizione ma con contenuto diverso fra le due matrici. Per esempio, se 12 celle su 64 differiscono, la conformance è $1 - 12/64 = 0.8125$. È un calcolo puramente combinatorio, molto più leggero del token replay — ma anche

più grossolano, perché non tiene conto delle frequenze reali delle tracce. Il confronto tra footprint permette tre usi: log vs modello (conformance vera e propria), modello vs modello (quanto due modelli si somigliano), log vs log (rileva il *concept drift*, come cambia il modo di lavorare nel tempo).

Da dire all'orale — footprint matrix

Tabella attività×attività con la relazione per cella: $\rightarrow/\leftarrow$ (causalità), # (mai adiacenti), || (parallelismo), dedotte dal directly-follows $>_L$. Antisimmetrica per $\rightarrow$, simmetrica per # e ||. Usata sia nell' α -algorithm sia nella conformance: $\text{conf} = 1 - d/n$ (celle discordanti su totali).

10.2 Alpha algorithm

Domanda ×2

Descrivi i passi dell' α -algorithm.

Contestualizzerei: l' α -algorithm è uno dei primi algoritmi di process discovery capaci di gestire correttamente la concorrenza. Prende un event log e ne ricava un Petri net (idealmente una sound workflow net), con strategia *play-in*: scandisce il log per estrarre le log-based ordering relations (la footprint), e con quelle costruisce la rete. L'idea di fondo è che ogni place della rete corrisponde a un “punto di decisione” tra un gruppo di attività che lo alimentano e un gruppo che ne consuma.

Definizione — I passi dell' α -algorithm

Dato un log L , si costruisce $\alpha(L) = (P_L, T_L, F_L, i_L)$:

1. T_L — una transizione per ogni attività che compare nel log.
2. T_I — le attività che compaiono come prima (first) di almeno una trace: gli eventi iniziali.
3. T_O — le attività che compaiono come ultima (last) di almeno una trace: gli eventi finali.
4. X_L — le coppie di insiemi (A, B) tali che ogni $a \in A$ è in causalità con ogni $b \in B$ ($a \rightarrow_L b$), tutte le attività di A sono mutuamente esclusive fra loro ($\#_L$), e così tutte quelle di B . Sono i candidati “punti di decisione”.
5. Y_L — si tengono solo le coppie massimali di X_L (quelle non contenute in un'altra): A e B i più grandi possibile.
6. P_L — un place $p_{(A,B)}$ per ogni coppia in Y_L , più il place iniziale i_L e quello finale o_L .
7. F_L — gli archi: da ogni $a \in A$ al place $p_{(A,B)}$ e dal place a ogni $b \in B$; inoltre da i_L verso ogni transizione iniziale (T_I) e da ogni transizione finale (T_O) verso o_L .
8. $\alpha(L)$ — la rete risultante, con marcatura iniziale un token in i_L .

Spiegherei che i passi 4 e 5 sono il cuore dell'algoritmo. Il passo 4 cerca ogni possibile “gruppo di cause $A \rightarrow$ gruppo di effetti B ” compatibile con la footprint: le attività dentro A devono essere mutuamente esclusive (perché convergono tutte sullo stesso place, quindi sono alternative), e così quelle in B ; e ogni a deve causare ogni b . Il passo 5 tiene solo i gruppi massimali, perché un place che collega insiemi più ampi cattura meglio la struttura: un place per $(\{a\}, \{b, e\})$ è più informativo di due place separati per $(\{a\}, \{b\})$ e $(\{a\}, \{e\})$.

Concluderei con i limiti, che il professore apprezza perché mostrano cosa un semplice conteggio di adiacenze non cattura:

- Dipendenze implicite: place “ridondanti” che vincolano scelte lontane non vengono ricostruiti (la footprint guarda solo le adiacenze dirette).
- Short loop: i cicli corti (come b, b , o cicli di lunghezza due) confondono l'algoritmo, perché

$a \parallel_L a$ non è distinguibile da un vero parallelismo.

- Noise: l'algoritmo non tiene conto delle frequenze, quindi un comportamento raro pesa quanto uno comune e non c'è modo di ignorare le tracce infrequenti (il rumore).

Da dire all'orale — α -algorithm

8 passi: T_L (transizioni), T_I/T_O (attività iniziali/finali), X_L (coppie (A, B) con $A \rightarrow B$, A e B internamente $\#$), Y_L (coppie massimali), P_L (un place per coppia + i_L, o_L), F_L (archi), rete finale. Cuore: passi 4–5. Limiti: dipendenze implicite, short loop, noise (ignora le frequenze).

Capitolo 11

Analisi quantitativa

Cambio di registro: dalle domande qualitative (soundness, deadlock) a quelle quantitative (quanto è veloce, quanto costa). Little's law e la Cycle Time Efficiency sono le due domande più frequenti.

11.1 Little's Law

Domanda ×2

Enuncia e spiega la Little's Law.

Introdurrei le tre grandezze in gioco, perché la legge le collega. In un processo si osservano: l'arrival rate λ (numero medio di nuovi casi creati per unità di tempo), il Work-In-Process WIP (numero medio di casi *attivi*, cioè non ancora completati, presenti in un dato istante), e il cycle time CT (tempo medio per gestire un caso dall'inizio alla fine).

Teorema — Little's Law

In un sistema stabile (il numero di casi attivi non cresce all'infinito):

$$WIP = \lambda \cdot CT$$

equivalentemente

$$CT = \frac{WIP}{\lambda}$$

Il punto che rende la legge notevole, e che va sottolineato, è la sua universalità: non serve sapere *nulla* della struttura interna del processo — niente sequenze, gateway, tempi delle singole attività. Bastano due grandezze osservabili dall'esterno: contare quanti casi arrivano (per λ) e contare quanti casi sono attivi in media (per WIP). Fu formulata da John Little nel 1954 e dimostrata nel 1961, e vale per qualsiasi sistema a coda stabile.

Un esempio numerico rende la cosa concreta: se in un anno di 250 giorni lavorativi arrivano 2500 domande, allora $\lambda = 10$ domande/giorno. Se campionando nel tempo si osservano in media $WIP = 200$ domande attive contemporaneamente, allora

$$CT = \frac{WIP}{\lambda} = \frac{200}{10} = 20 \text{ giorni}$$

e abbiamo ottenuto il cycle time medio senza mai aver cronometrato una singola domanda.

Chiuderei con la lettura operativa, che è ciò che interessa a un'azienda. Dalla relazione $WIP = \lambda \cdot CT$: se l'arrival rate λ aumenta (più richieste in arrivo) e non si vuole far crescere il WIP (cioè non si vuole impegnare più personale o spazio per gestire più casi contemporaneamente), l'unica leva rimasta è ridurre il cycle time CT — cioè lavorare più in fretta, snellendo il processo. La legge quindi non è solo descrittiva: indica *dove* intervenire.

Da dire all'orale — Little's Law

$WIP = \lambda \cdot CT$ (in un sistema stabile). λ = tasso di arrivo, WIP = casi attivi in media, CT = cycle time. Universale: non serve la struttura interna, solo due grandezze osservabili dall'esterno. Lettura: se λ sale e il WIP deve restare fisso, bisogna ridurre il CT .

11.2 Cycle Time Efficiency

Domanda ×2

Cos'è la Cycle Time Efficiency: definizione e in quale intervallo varia ($[0, 1]$).

Motiverei la nozione partendo dal fatto che il cycle time, da solo, non dice *dove* si nasconde il tempo perso. Conviene scomporlo in due parti: il waiting time (la parte in cui *nessuno* lavora sul caso — documenti in transito, attesa di un attore libero, code) e il processing time (il tempo in cui qualcuno *lavora davvero* sul caso). In molti processi reali il waiting time è la componente dominante: lavorazioni a lotti, attori non disponibili, approvazioni che aspettano una firma.

Per quantificare quanto “tempo utile” c'è nel cycle time si introduce il theoretical cycle time e poi il rapporto:

Definizione — Theoretical cycle time e Cycle Time Efficiency

Il theoretical cycle time TCT si calcola con le *stesse formule* del cycle time (somma per la sequenza, media pesata per lo XOR, massimo per l'AND, ...), ma usando i processing time al posto dei cycle time: è il tempo che il processo impiegherebbe se non ci fosse *mai* attesa.

La Cycle Time Efficiency è

$$CTE = \frac{TCT}{CT}$$

Sull'intervallo, che è la parte esplicitamente richiesta: poiché il processing time è sempre una *parte* del cycle time (il tempo di lavoro effettivo non può superare il tempo totale, che include anche l'attesa), vale sempre $TCT \leq CT$, e quindi

$$CTE \in [0, 1].$$

L'interpretazione dei due estremi:

- CTE vicino a 1: quasi tutto il cycle time è lavoro effettivo, c'è pochissima attesa. Resta poco margine di miglioramento senza cambiamenti radicali del processo.
- CTE vicino a 0: il cycle time è quasi tutto attesa, il lavoro effettivo è una frazione minima. C'è molto margine di miglioramento riducendo il waiting time (per esempio eliminando code e passaggi di consegne inutili).

In pratica il CTE è un “termometro” dell'efficienza temporale: dice quanto del tempo che un caso passa nel sistema è effettivamente produttivo. Un valore basso è un campanello d'allarme che indica dove intervenire (sull'attesa, non sulla velocità di lavoro).

Da dire all'orale — Cycle Time Efficiency

$CTE = TCT/CT$, dove TCT usa i *processing time* (nessuna attesa) nelle stesse formule del cycle time. Poiché $\text{processing} \leq \text{cycle time}$, $CTE \in [0, 1]$. Vicino a 1: poco da migliorare; vicino a 0: molta attesa, ampio margine riducendo il waiting time.